Methodology

Task1: Cheater Detection

1.1 EDA

For Task 1 we aim to predict whether a player is a cheater based on their gameplay statistics. In the EDA cells we first loaded the training and test sets and inspected their shapes to understand how many samples and features we have:

```
train = pd.read_csv("train.csv")
test  = pd.read_csv("test.csv")
print("Train shape:", train.shape)
print("Test shape:", test.shape)
```

```
TARGET = "is_cheater"
ID_COL = "id"
feature_cols = [c for c in train.columns if c not in [TARGET, ID_COL]]
```

Target and ID: *is_cheater* as the binary target and id as the unique identifier.

Feature columns: all remaining columns used as predictors.

We then split the feature set into numeric and categorical columns to plan different preprocessing strategies:

```
num_cols = train[feature_cols].select_dtypes(include=[np.number]).columns.tolist()
cat_cols = train[feature_cols].select_dtypes(exclude=[np.number]).columns.tolist()
print("Numeric columns:", len(num_cols))
print(num_cols)
print("\nCategorical columns:", len(cat_cols))
print(cat_cols)
```

1.2 Data Preprocessing

Feature selection: From the above section, we already removed id and the target *is_cheater* from the feature set so they would not be used as inputs.

The preprocessing section prepares the data for the models:

Handling missing numerical values.

Numeric features: imputed using the mean of each column:

```
num_imputer = SimpleImputer(strategy="mean")
train[num_cols] = num_imputer.fit_transform(train[num_cols])
test[num_cols]  = num_imputer.transform(test[num_cols])
```

Handling missing categorical values.

Categorical features: imputed using the most frequent category:

```
if len(cat_cols) > 0:
    cat_imputer = SimpleImputer(strategy="most_frequent")
    train[cat_cols] = cat_imputer.fit_transform(train[cat_cols])
    test[cat_cols]  = cat_imputer.transform(test[cat_cols])
for col in cat_cols:
    le = LabelEncoder()
    all_labels = pd.concat([train[col], test[col]]).unique()
    le.fit(all_labels)
    train[col] = le.transform(train[col])
    test[col]  = le.transform(test[col])
```

Encoding categorical variables: each categorical column was label-encoded so that categories are mapped to integer indices. The encoder was fit on the combined train+test values of each column to ensure consistent mapping between datasets.

Feature scaling: all numeric features were standardized using z-score normalization This keeps features on a similar scale and helps tree ensembles and other models behave more stably:

```
scaler = StandardScaler()
train[num_cols] = scaler.fit_transform(train[num_cols])
test[num_cols]  = scaler.transform(test[num_cols])
```

Final matrices: after preprocessing, we constructed:

```
X_train = train[feature_cols]
y_train = train[TARGET]
X_test  = test[feature_cols]
```

1.3 Model Design

We experimented with two related LightGBM-based designs: a baseline model and an advanced model that incorporates the hints from the slides.

1.3.1 Baseline model

Performed a train/validation split (e.g., 80/20) on the cleaned training data, stratified by *is_cheater* to preserve the class ratio in both sets:

```
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_cleaned, y_train_cleaned,
    test_size=0.2,
    random_state=42,
    stratify=y_train_cleaned
)
```

Trained a single LightGBM classifier with standard hyperparameters (number of trees, learning rate, maximum leaves, subsampling, and feature subsampling):

```
model = lgb.LGBMClassifier(
    n_estimators=800,
    learning_rate=0.03,
    num_leaves=64,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)
model.fit(X_tr, y_tr)
```

This gave us an initial benchmark for accuracy, F1 score, and ROC AUC.

1.3.2 Advanced model (final approach)

To better reflect the assignment hints, we refined the design.

Class imbalance handling: we computed class weights from the training labels and passed them to LightGBM so that misclassifying cheaters is penalized more appropriately:

```
classes = np.unique(y_train_cleaned)
class_weights = compute_class_weight(class_weight="balanced", classes=classes, y=y_train_cleaned)
class_weight_dict = {int(c): float(w) for c, w in zip(classes, class_weights)}
print("Class weights:", class_weight_dict)
```

Cross-validation: we used Stratified K-Fold (e.g., 5 folds) on X_train and y_train. For each fold a new LightGBM model was trained on the training portion and evaluated on the validation portion:

```
n_splits = 5
skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
```

Threshold tuning: instead of always using a 0.5 probability cutoff, we scanned possible thresholds and selected the one that maximized F1 score on each fold:

```
oof_proba = np.zeros(len(X_train_cleaned))
test_proba = np.zeros(len(X_test))
best_thresholds = []
```

Ensembling: for the test set we averaged the predicted probabilities from all folds to obtain a more robust prediction, then applied the tuned threshold (or the mean of the best thresholds across folds) to convert probabilities to final cheater/non-cheater labels:

```
for fold, (tr_idx, val_idx) in enumerate(skf.split(X_train_cleaned, y_train_cleaned), 1):
    print(f"\n===== Fold {fold}/{n_splits} ===telek_p")
    X_tr, X_val = X_train_cleaned.iloc[tr_idx], X_train_cleaned.iloc[val_idx]
    y_tr, y_val = y_train_cleaned.iloc[tr_idx], y_train_cleaned.iloc[val_idx]
```

This advanced design focuses on F1 performance on the cheater class and reduces variance through cross-validation and ensembling.

1.4 Evaluation & Results

```
===== Fold 1/5 ===telek_p
[LightGBM] [Info] Number of positive: 27303, number of negative: 50895
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.023939 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 7949
[LightGBM] [Info] Number of data points in the train set: 78198, number of used features: 33
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499999 -> initscore=-0.000003
[LightGBM] [Info] Start training from score -0.000003
  Fold 1 best threshold: 0.500
  Fold 1 Accuracy: 0.7873
  Fold 1 F1:       0.7257
  Fold 1 ROC AUC:  0.8954

===== Fold 2/5 ===telek_p
[LightGBM] [Info] Number of positive: 27303, number of negative: 50895
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.023265 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 7949
[LightGBM] [Info] Number of data points in the train set: 78198, number of used features: 33
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499999 -> initscore=-0.000003
[LightGBM] [Info] Start training from score -0.000003
  Fold 2 best threshold: 0.500
  Fold 2 Accuracy: 0.7870
  Fold 2 F1:       0.7247
  Fold 2 ROC AUC:  0.8956

===== Fold 3/5 ===telek_p
[LightGBM] [Info] Number of positive: 27303, number of negative: 50895
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.022941 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 7949
[LightGBM] [Info] Number of data points in the train set: 78198, number of used features: 33
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499999 -> initscore=-0.000003
[LightGBM] [Info] Start training from score -0.000003
  Fold 3 best threshold: 0.500
  Fold 3 Accuracy: 0.7902
  Fold 3 F1:       0.7287
  Fold 3 ROC AUC:  0.8991
```

```
===== Fold 4/5 ===telek_p
[LightGBM] [Info] Number of positive: 27304, number of negative: 50895
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.055485 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 7949
[LightGBM] [Info] Number of data points in the train set: 78199, number of used features: 33
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500008 -> initscore=0.000033
[LightGBM] [Info] Start training from score 0.000033
  Fold 4 best threshold: 0.450
  Fold 4 Accuracy: 0.7660
  Fold 4 F1:       0.7165
  Fold 4 ROC AUC:  0.8908

===== Fold 5/5 ===telek_p
[LightGBM] [Info] Number of positive: 27303, number of negative: 50896
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.023652 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 7949
[LightGBM] [Info] Number of data points in the train set: 78199, number of used features: 33
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499994 -> initscore=-0.000023
[LightGBM] [Info] Start training from score -0.000023
  Fold 5 best threshold: 0.500
  Fold 5 Accuracy: 0.7894
  Fold 5 F1:       0.7274
  Fold 5 ROC AUC:  0.8977

Average best threshold across folds: 0.49000000000000005

===== Overall CV Metrics (OOF) ===telek_p
  Accuracy: 0.7841
  F1 Score: 0.7248
  ROC AUC:  0.8957
```

For each model we evaluated performance on the validation data.

Metrics for the baseline: accuracy, F1 score, and ROC AUC using a fixed threshold of 0.5. These results served as our reference point.

Metrics for the advanced model: for every fold we computed:

F1 score using the best threshold found for that fold.

ROC AUC using predicted probabilities.

Accuracy for completeness.

We then reported the average metrics across folds as the final estimate of generalization performance. In our experiments, the advanced model slightly traded off raw accuracy but improved F1 score compared with the baseline, which is more important for detecting cheaters.

The final predictions for the competition/test set were generated from the cross-validated ensemble and written to a submission file containing id and the predicted is_cheater label.

1.5 Insights gained from model behavior

Class weighting and threshold tuning matter: simply using a default 0.5 threshold and unweighted loss led to good ROC AUC but lower F1 on cheaters. Adjusting for class imbalance and explicitly tuning the threshold improved the balance between precision and recall.

Cross-validation is more reliable than a single split: performance metrics across folds were more stable and gave a better sense of how the model would behave on unseen players.

Feature preprocessing is essential: after imputation and scaling, the model trained faster and produced more consistent validation results.

1.6 Common mistakes or failed experiments

Naïve baseline without imbalance handling: the very first version of the model used no class weights and relied on accuracy, which underestimated how often cheaters were missed.

Fixed threshold at 0.5: early results showed that this default threshold was not optimal for F1; scanning thresholds demonstrated that a slightly lower or higher cutoff can significantly improve the balance of precision and recall.

1.7 Domain interpretation of the results

The final cheater-detection model can prioritize suspicious accounts for manual review or automated actions. A tuned threshold allows game operators to decide whether they want to be more aggressive (catch more cheaters at the cost of more false positives) or more conservative.

Features derived from gameplay behavior (accuracy, activity patterns, etc.) appear sufficient to distinguish many cheaters from normal players, demonstrating that data-driven anti-cheat systems are feasible.

Integrating this model into the live system could reduce the impact of cheaters on honest players, improve game fairness, and protect in-game economies.

Task2: Player Segmentation

2.1 EDA

In Task 2 the goal is to assign each player to a behavior segment. We first loaded the training and test data and checked basic properties such as number of rows/columns and presence of missing values. Rows in the training set without a known segment label were dropped so that only fully labeled players were used for supervised learning.

```
print("🔥 Initiating God Mode Pipeline...")
print("📦 Loading Data...")
train_df = pd.read_csv('train.csv')
test_df = pd.read_csv('test.csv')

test_ids = test_df['id']
train_df = train_df.dropna(subset=['segment']).reset_index(drop=True)
y = train_df['segment'].astype(int)
train_len = len(train_df)
```

Exploratory checks focused on:

The class distribution of segment using value counts, to understand how imbalanced the four segments are.

Basic statistics of spending, playtime, social features, and progression features, to see how they differ between segments.

Patterns of missing values in both numeric and categorical columns, which later guided our imputation strategy.

These observations confirmed that the four segments are moderately imbalanced and that many of the financial, engagement, and social variables contain missing entries that must be handled carefully.

2.2 Data Preprocessing

The preprocessing pipeline for Task 2 is designed so that feature engineering can be applied consistently to both train and test sets and so that CatBoost can handle categorical variables effectively.

Combine train and test for feature engineering

From the original *train_df*, we removed the segment column and concatenated the remaining columns with *test_df* to form a single dataframe *df_all*.

```
df_all = pd.concat([train_df.drop(columns=['segment']), test_df], axis=0).reset_index(drop=True)
df_all = df_all.drop(columns=['id', 'player_id'])
```

Identifier columns id and *player_id* were dropped because they do not carry behavioral semantics for segmentation.

Feature engineering base input (numeric imputation first)

Only numerical columns were imputed using a median imputer.

```
num_cols = df_all.select_dtypes(include=[np.number]).columns
imputer = SimpleImputer(strategy='median')
df_all_filled_num = df_all.copy()
df_all_filled_num[num_cols] = imputer.fit_transform(df_all[num_cols])
```

A copy of df_all with imputed numeric values was passed into an ***extreme_fe*** function (described in the next subsection) to create higher-level behavioral features.

```
df_fe = extreme_fe(df_all_filled_num)
```

Merging engineered features back

New columns created by ***extreme_fe*** were identified and concatenated back to the original *df_all*. This keeps original raw features plus additional, more interpretable aggregates.

```
new_cols = [c for c in df_fe.columns if c not in df_all.columns]
df_all = pd.concat([df_all, df_fe[new_cols]], axis=1)
```

Categorical preprocessing for CatBoost

```
cat_features = [
    'region', 'platform', 'device_type', 'payment_method', 'language',
    'account_status', 'player_type_tag', 'engagement_level',
    'loyalty_tier', 'skill_tier', 'preferred_game_mode',
    'competitive_rank', 'vip_tier'
]
```

A list of categorical feature names was defined (region, platform, device type, payment method, language, account status, player type tag, engagement level, loyalty tier, skill tier, preferred game mode, competitive rank, VIP tier, etc.).

```
for col in cat_features:
    if col in df_all.columns:
        df_all[col] = df_all[col].fillna('Unknown').astype(str)
```

For each of these columns present in *df_all*, missing values were filled with the string "Unknown" and the column was cast to str. This is the format expected by CatBoost's native categorical handling.

Final matrices:

```
X = df_all.iloc[:train_len]
X_test = df_all.iloc[train_len:]
```

2.3 Model Design

We framed segmentation as a 4-class classification problem and used CatBoostClassifier because it naturally supports categorical features and handles complex tabular interactions well.

Feature engineering (***extreme_fe***)

The function ***extreme_fe(df)*** creates several domain-inspired features grouped into five conceptual categories:

```python
def extreme_fe(df):
    df = df.copy()

    # --- Group 1: Financial Power (จับ Whale) ---
    # Log Transform เงิน (เพราะพวก Whale จ่ายหนักเว่อร์ กราฟจะเบ้)
    df['log_total_spending'] = np.log1p(df['total_spending_thb'])
    df['log_avg_monthly'] = np.log1p(df['avg_monthly_spending'])
    # Spending Rate: จ่ายหนักแค่ไหนเทียบกับอายุไอดี
    df['spending_per_day'] = df['total_spending_thb'] / (df['account_age_days'] + 1)
    # Spending Efficiency: จ่ายคุ้มไหม (จ่ายเทียบกับเวลาเล่น)
    df['spending_per_hour'] = df['total_spending_thb'] / (df['total_playtime_hours'] + 1)
    # Wallet Depth: ศักยภาพเปย์ (VIP Tier * Monthly Spending)
    df['wallet_power'] = df['vip_tier'] * df['avg_monthly_spending']
    # --- Group 2: Grinding Intensity (จับ Grinder) ---
    # Dedication: ความทุ่มเท (เล่นนาน * ความถี่)
    df['dedication_score'] = df['avg_session_duration'] * df['play_frequency']
    # Hardcore Ratio: เล่น Rank เยอะแค่ไหนเทียบกับการเล่นทั้งหมด (สมมติ Rank คือส่วนหนึ่งของ total)
    # ถ้าไม่มี total matches ใช้ proxy อื่นแทน
    df['rank_intensity'] = df['ranked_participation_rate'] * df['win_rate_ranked']
    # Progression Speed: เวลไวแค่ไหน
    df['level_up_efficiency'] = df['speed_of_progression'] * df['achievement_completion_rate']
    # --- Group 3: Social Connectivity (จับ Social) ---
    # Social Network Density: เพื่อนเยอะ + เล่นทีมเยอะ
    df['social_density'] = df['friend_count'] * df['team_play_percentage']
    # Giver Index: สายเปย์เพื่อน (ส่งของขวัญ / จำนวนเพื่อน)
    df['generosity_ratio'] = df['gifts_sent_received'] / (df['friend_count'] + 1)
    # Chatty Score: คุยเก่งแค่ไหนเทียบกับเวลาเล่น
    df['chat_per_hour'] = df['chat_activity_score'] / (df['avg_session_duration'] + 1)
    # --- Group 4: Loyalty & Consistency ---
    # Loyalty Score: อยู่มานาน + ล็อกอินต่อเนื่อง
    df['loyalty_score'] = df['account_age_days'] * df['login_streak']
    # Dormancy Risk: ไม่ได้ล็อกอินนานแค่ไหนเทียบกับอายุไอดี
    df['dormancy_ratio'] = df['days_since_last_login'] / (df['account_age_days'] + 1)
    # --- Group 5: Preference Profiling ---
    # ชอบแต่งตัว vs ชอบเก่ง
    df['cosmetic_vs_performance'] = df['item_type_preference_cosmetic'] - df['item_type_preference_performance']
    return df
```

These features encode interpretable patterns that correspond closely to how game designers think about player types.

CatBoost configuration:

```python
cat_params = {
    'loss_function': 'MultiClass',
    'eval_metric': 'TotalF1',
    'iterations': 1000,
    'learning_rate': 0.1,
    'depth': 6,
    'l2_leaf_reg': 3,
    'auto_class_weights': 'Balanced',
    'early_stopping_rounds': 50,
    'cat_features': [c for c in cat_features if c in X.columns],
    'verbose': 100,
    'random_seed': 42,
    'task_type': 'CPU',
    'allow_writing_files': False
}
```

Categorical columns are passed via the ***cat_features*** argument so that CatBoost can use its internal encoding.

Cross-validation strategy

Used StratifiedKFold with ***n_splits*** = 5, shuffling and a fixed random seed to preserve class proportions in each fold.

```
n_folds = 3
skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=42)
n_folds = 5
skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=42)
oof_preds = np.zeros((len(X), 4))
test_preds_prob = np.zeros((len(X_test), 4))
feature_importance_list = []
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    print(f"\n❌  FOLD {fold+1}/{n_folds} STARTED")
    X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_tr, y_val = y.iloc[train_idx], y.iloc[val_idx]
    train_pool = Pool(X_tr, y_tr, cat_features=[c for c in cat_features if c in X.columns])
    val_pool = Pool(X_val, y_val, cat_features=[c for c in cat_features if c in X.columns])
    model = CatBoostClassifier(**cat_params)
    model.fit(train_pool, eval_set=val_pool, use_best_model=True)
    val_probs = model.predict_proba(val_pool)
    oof_preds[val_idx] = val_probs
    test_preds_prob += model.predict_proba(X_test) / n_folds
    val_preds_cls = np.argmax(val_probs, axis=1)
    f1 = f1_score(y_val, val_preds_cls, average='macro')
    print(f"✅ Fold {fold+1} F1-Macro: {f1:.4f}")
    feature_importance_list.append(model.get_feature_importance())
```

For each fold:

Fitted a CatBoost model on the training split and validated on the hold-out fold.

Stored predicted probabilities for validation samples (OOF predictions) and accumulated test set probabilities for ensembling.

This design focuses on robust generalization, handling class imbalance, and leveraging rich feature interactions.

2.4 Evaluation & Results

Model performance was evaluated using macro-averaged F1 score, which treats all four segments equally regardless of their frequency.

For each fold in cross-validation:

```
⚔ FOLD 1/5 STARTED
0:  learn: 0.5839891    test: 0.5841268 best: 0.5841268 (0) total: 1.31s    remaining: 21m 53s
100:    learn: 0.7484197    test: 0.7451280 best: 0.7452694 (95)    total: 2m 34s   remaining: 22m 58s
200:    learn: 0.7646147    test: 0.7562791 best: 0.7567219 (198)   total: 5m 8s    remaining: 20m 26s
300:    learn: 0.7723404    test: 0.7573660 best: 0.7583250 (288)   total: 7m 39s   remaining: 17m 47s
400:    learn: 0.7788369    test: 0.7582779 best: 0.7592787 (388)   total: 10m 15s  remaining: 15m 20s
Stopped by overfitting detector  (50 iterations wait)

bestTest = 0.7592786987
bestIteration = 388

Shrink model to first 389 iterations.
✅ Fold 1 F1-Macro: 0.7517

⚔ FOLD 2/5 STARTED
0:  learn: 0.5839698    test: 0.5822078 best: 0.5822078 (0) total: 1.28s    remaining: 21m 17s
100:    learn: 0.7477471    test: 0.7403830 best: 0.7403830 (100)   total: 2m 35s   remaining: 22m 59s
200:    learn: 0.7652713    test: 0.7514179 best: 0.7515485 (199)   total: 5m 5s    remaining: 20m 16s
Stopped by overfitting detector  (50 iterations wait)

bestTest = 0.7525633894
bestIteration = 216

Shrink model to first 217 iterations.
✅ Fold 2 F1-Macro: 0.7451

⚔ FOLD 3/5 STARTED
0:  learn: 0.6131447    test: 0.6079053 best: 0.6079053 (0) total: 1.26s    remaining: 21m 4s
100:    learn: 0.7493185    test: 0.7375144 best: 0.7375144 (100)   total: 2m 36s   remaining: 23m 13s
200:    learn: 0.7673680    test: 0.7490778 best: 0.7494459 (183)   total: 5m 7s    remaining: 20m 22s
300:    learn: 0.7744774    test: 0.7506690 best: 0.7515921 (290)   total: 7m 36s   remaining: 17m 40s
Stopped by overfitting detector  (50 iterations wait)

bestTest = 0.7515920778
bestIteration = 290
```

We computed predicted probabilities on the validation split and converted them to class labels via argmax.

Fold-level F1-macro scores were printed to monitor stability across folds.

After completing all folds Out-of-fold (OOF) predictions were aggregated for all training samples, and we computed:

```
Classification Report (OOF):
              precision    recall  f1-score   support

           0       0.84      0.77      0.80     40064
           1       0.72      0.76      0.74     25397
           2       0.69      0.73      0.71     20549
           3       0.72      0.76      0.74     15648

    accuracy                           0.76    101658
   macro avg       0.74      0.75      0.75    101658
weighted avg       0.76      0.76      0.76    101658
```

Overall macro F1 on the entire training set using OOF labels.

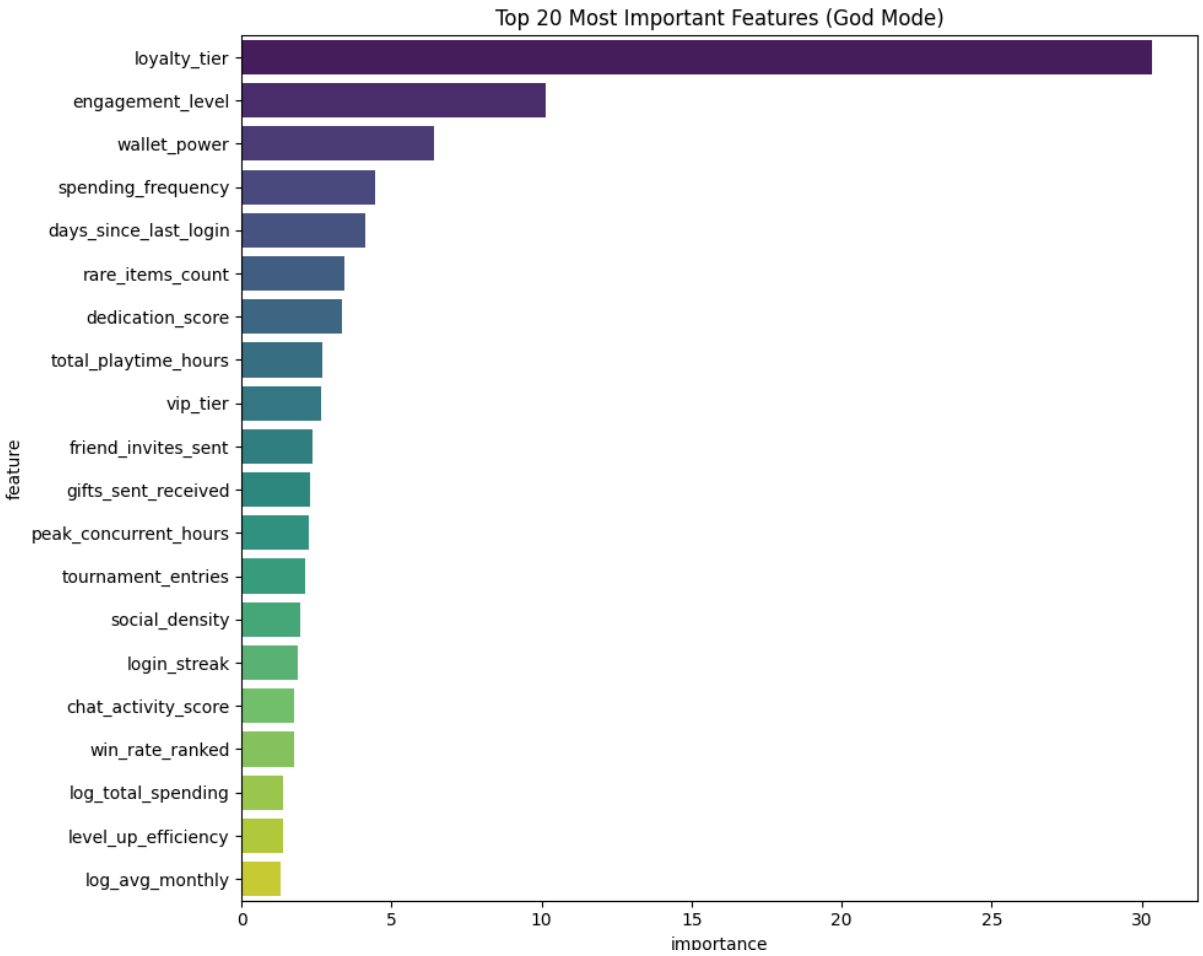
A full classification report (per-class precision, recall, F1, and support) to see how well each segment was captured.

```
Prediction Distribution:
segment
0    0.362818
1    0.260458
2    0.215806
3    0.160918
```

For the test set, probabilities from each fold were averaged to obtain final class probabilities. Final segment predictions were again chosen via argmax and saved into a submission file along with player IDs.

We also computed and plotted feature importance (mean importance across folds) and saved a bar chart of the top 20 most influential features.

This evaluation setup gives a realistic estimate of performance and highlights which engineered features contribute most to segmentation.



2.5 Insights gained from model behavior

Using TotalF1 / macro F1 as the main metric encourages the model to perform well on all segments, not just the majority class.

Financial power, grinding intensity, and social connectivity features consistently appear among the top contributors, aligning with the intuition that monetization, engagement, and social behavior define different player types.

The combination of raw features and engineered aggregates allowed CatBoost to learn more nuanced boundaries between segments, especially for mid-range or ambiguous players.

2.6 Common mistakes or failed experiments

Ignoring feature engineering: using only raw numeric and categorical features led to weaker performance and less interpretable results; the extreme feature engineering step notably improved macro F1.

Using default loss/metrics: relying on accuracy or default settings would under-emphasize minority segments, whereas explicitly optimizing TotalF1 gives better balance.

Insufficient cross-validation: a simple train/validation split can give misleading results depending on how segments are distributed; stratified K-fold CV provides more stable evaluation.

Overly long training without early stopping: very large iteration counts slowed training and risked overfitting; adding early stopping and reasonable iteration limits improved both speed and generalization.

2.7 Domain interpretation of the results

Personalized live-ops and marketing: different segments (e.g., whales, grinders, social players, casuals) can receive tailored in-game offers, events, or messaging.

Game design feedback: understanding which features are most important in distinguishing segments helps designers see which mechanics drive long-term engagement, spending, or social interaction.

Retention and monetization strategy: segments with high churn risk but strong potential value (for example, grinders who rarely spend) can be targeted with specific retention campaigns or monetization experiments.

3. Task 3: Next 30 Days Spending Prediction

3.1 EDA

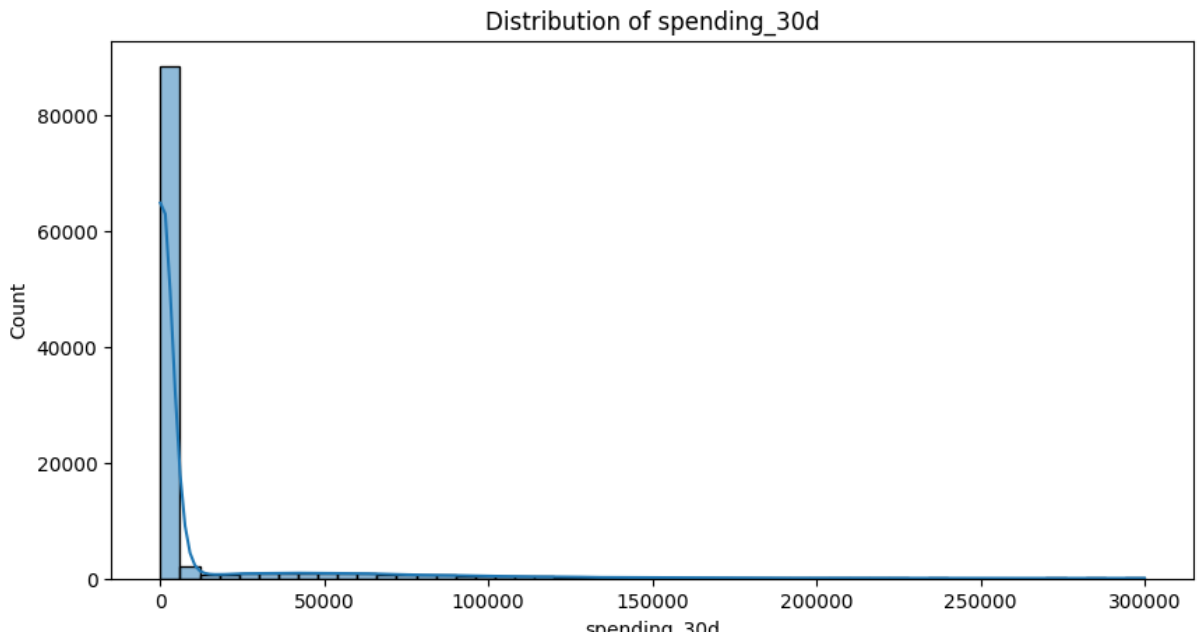
In Task 3 the goal is to predict how much each player will spend in the next 30 days (a continuous value in THB).

Checked the shape of the train and test sets and confirmed that the train set has one additional column: the target ***spending_30d*** :

```
train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")

train.shape, test.shape
```

```
print(train.head())
print("\n--- INFO ---")
print(train.info())
print("\n--- Missing Values ---")
print(train.isna().sum())
print("\n--- Describe (numerical) ---")
print(train.describe())
```

Looked at the distribution of spending_30d and found it to be heavily skewed:

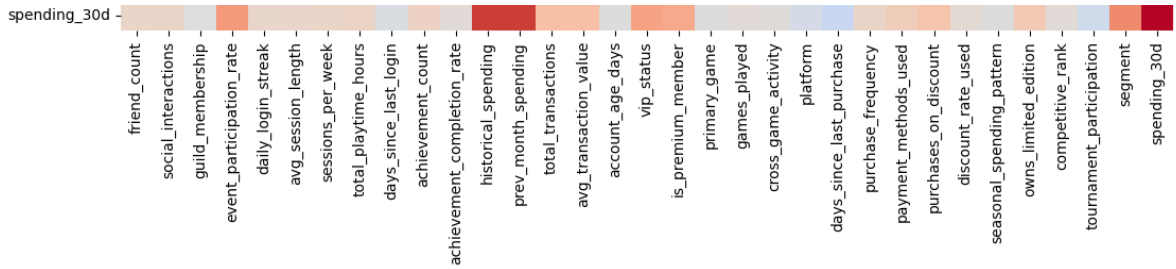


Almost half of the players spend exactly 0 THB in the next 30 days.

A small portion of players spend very large amounts (long right tail).

Calculated correlations between spending_30d and other features. The strongest positive correlations were found with:

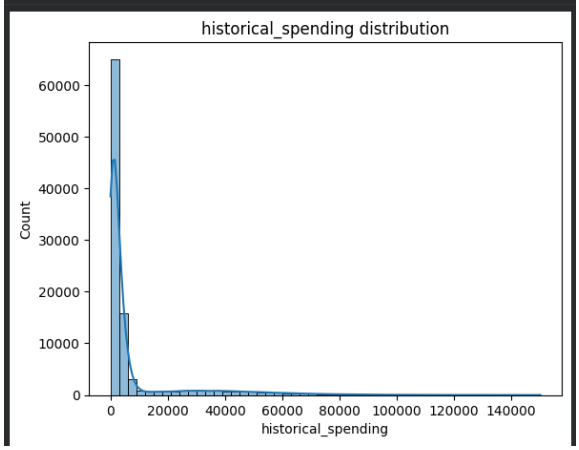
```
plt.figure(figsize=(16,12))
sns.heatmap(train.drop(['id', 'player_id'], axis=1).corr(), cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
plt.show()

train.drop(['id', 'player_id'], axis=1).corr()['spending_30d'].sort_values(ascending=False).head(15)
```



	spending_30d
spending_30d	1.000000
historical_spending	0.868204
prev_month_spending	0.865581
segment	0.568803
event_participation_rate	0.497855
vip_status	0.447114
is_premium_member	0.417903
avg_transaction_value	0.287226
total_transactions	0.286096
purchases_on_discount	0.250172
owns_limited_edition	0.209493
payment_methods_used	0.179597
achievement_count	0.136591
total_playtime_hours	0.121691
purchase_frequency	0.117925

Historical spending and previous-month spending:

```
sns.histplot(train['historical_spending'], bins=50, kde=True)
plt.title("historical_spending distribution")
plt.show()
```



Inspected missing values and observed:

```
print("\n--- Missing Values ---")
print(train.isna().sum())
```

Several important behavioral features contain missing entries.

```
--- Missing Values ---
id                              0
player_id                       0
friend_count                 6046
social_interactions          6106
guild_membership             6028
event_participation_rate     6001
daily_login_streak           6107
avg_session_length           6028
sessions_per_week            6162
total_playtime_hours         6132
days_since_last_login        6040
achievement_count            6011
achievement_completion_rate  6061
historical_spending          6099
prev_month_spending          5969
total_transactions           6133
avg_transaction_value        6031
account_age_days             6102
vip_status                   6094
is_premium_member            6021
primary_game                 6168
games_played                 5985
cross_game_activity          6083
platform                     6095
days_since_last_purchase     6148
purchase_frequency           5995
payment_methods_used         6108
purchases_on_discount        5886
discount_rate_used           5945
seasonal_spending_pattern    6066
owns_limited_edition         6075
competitive_rank             5950
tournament_participation     6049
segment                      6069
spending_30d                    0
dtype: int64
```

The segment column is missing for a non-trivial number of rows (a few thousand players), which would be problematic to drop.

From this EDA we concluded that the problem combines zero-inflation (many non-spenders) with highly skewed continuous outcomes, and that we need a careful strategy for missing values and the segment feature.

3.2 Data Preprocessing

The preprocessing pipeline for Task 3 has three main parts: handling missing values, reconstructing the segment feature, and transforming skewed features.

Handling missing values (excluding segment):

```
EXCLUDED_FROM_SIMPLE_IMPUTATION = ["id", "player_id", "spending_30d", "segment"]
categorical_cols = [
    "primary_game", "platform",
    "is_premium_member", "guild_membership"
]

categorical_cols = [c for c in categorical_cols if c in train.columns and c not in EXCLUDED_FROM_SIMPLE_IMPUTATION]
numeric_cols = [
    c for c in train.columns
    if c not in categorical_cols
    and c not in EXCLUDED_FROM_SIMPLE_IMPUTATION
]

num_imputer = SimpleImputer(strategy="median")
cat_imputer = SimpleImputer(strategy="most_frequent")
train[numeric_cols] = num_imputer.fit_transform(train[numeric_cols])
test[numeric_cols]  = num_imputer.transform(test[numeric_cols])
if len(categorical_cols) > 0:
    train[categorical_cols] = cat_imputer.fit_transform(train[categorical_cols])
    test[categorical_cols]  = cat_imputer.transform(test[categorical_cols])
```

The ID columns and the target spending_30d were excluded from imputation.

Numerical features (e.g., spending history, engagement counts, progression metrics) were imputed using a robust statistic such as the median

Categorical features (e.g., platform, primary game, premium membership flags) were imputed using the most frequent category.

After this step, all features except segment had no missing values.

Model-based imputation of segment

```
seg_known = train[train["segment"].notna()]
seg_missing = train[train["segment"].isna()]
print("Known segment rows:", len(seg_known))
print("Missing segment rows:", len(seg_missing))
feature_cols = [
    c for c in train.columns
    if c not in ["id", "player_id", "segment", "spending_30d"]
]
X = seg_known[feature_cols]
y = seg_known["segment"].astype(int)

clf = RandomForestClassifier(
    n_estimators=300,
    max_depth=12,
    random_state=42
)
clf.fit(X, y)
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
clf.fit(X_tr, y_tr)
preds = clf.predict(X_val)
print("\n=== Segment Prediction Model Report ===")
print(classification_report(y_val, preds))
```

The training data was split into two subsets: rows with known segment labels and rows with missing segment labels.

A separate classifier (for example a tree-based model) was trained to predict the segment from the cleaned gameplay features on the known subset.

This classifier achieved high accuracy and macro F1 on a validation split, showing that segment can be inferred reliably from other behavior.

The trained classifier was then used to fill in the missing segment values in the remaining rows, so that the main model could use segment as a complete feature rather than dropping those rows.

Transforming skewed features:

```
log_cols = [
    "historical_spending",
    "prev_month_spending",
    "avg_transaction_value",
    "tournament_participation",
    "owns_limited_edition",
    "cross_game_activity",
    "is_premium_member",
    "payment_methods_used",
]
log_cols = [c for c in log_cols if c in train.columns]
print("Log-transforming these columns:", log_cols)
for col in log_cols:
    train[col] = np.log1p(train[col])
    test[col]  = np.log1p(test[col])
```

Features representing monetary amounts, counts, or other highly skewed quantities (e.g., historical spending, previous-month spending, average transaction value, limited-edition ownership, cross-game activity) were transformed using a logarithmic transform (log1p).

The target variable was also transformed: a new ***spending_30d_log*** was created as the log1p of ***spending_30d***. This helps stabilize training and reduces the influence of extremely high spenders on the loss.

Some additional log transforms were applied to other count-like features to make their distributions more symmetric.

This preprocessing ensured a complete feature matrix with reduced skew and preserved useful information from the segment column.

3.3 Model Design

Because of the large number of zero spenders and the skewed positive values, we adopted a two-stage modeling approach instead of a single regression model.

3.3.1 Spender / non-spender classifier

```
train["is_spender"] = (train["spending_30d"] > 0).astype(int)

drop_cols = ["id", "player_id", "spending_30d", "spending_30d_log", "is_spender"]
features = [c for c in train.columns if c not in drop_cols]

print("Number of features:", len(features))

X_cls = train[features]
y_cls = train["is_spender"]
```

A binary target is_spender was defined: 1 if a player spends more than 0 THB in the next 30 days, otherwise 0.

All preprocessed features (including the imputed segment and log-transformed variables) were used as predictors, excluding the original target and IDs.

A tree-based classifier (e.g., LightGBM) was trained on an 80/20 stratified train–validation split:

```
Xc_train, Xc_val, yc_train, yc_val = train_test_split(
    X_cls, y_cls, test_size=0.2, random_state=42, stratify=y_cls
)

clf = lgb.LGBMClassifier(
    n_estimators=600,
    learning_rate=0.03,
    max_depth=-1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

clf.fit(Xc_train, yc_train)

val_proba = clf.predict_proba(Xc_val)[:, 1]
```

After training, a probability of being a spender was predicted for each player.

```
best_t = 0.5
best_f1 = -1

for t in np.linspace(0.1, 0.9, 17):
    preds = (val_proba > t).astype(int)
    score = f1_score(yc_val, preds)
    if score > best_f1:
        best_f1 = score
        best_t = t
```

Instead of fixing the probability threshold at 0.5, a range of thresholds was scanned and the threshold that maximized F1 score on the validation set was selected. This directly targets a good balance between correctly detecting spenders and avoiding false alarms.

3.3.2 Spending amount regressor (for spenders)

For the regression stage, only players with is_spender = 1 (actual spenders in the training data) were used. The target was the log-transformed spending spending_30d_log:

```
spender_mask = train["is_spender"] == 1
X_reg = train.loc[spender_mask, features]
y_reg = train.loc[spender_mask, "spending_30d_log"]
```

```
Xr_train, Xr_val, yr_train, yr_val = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

An ensemble of three regressors was used, for example:

One based on gradient boosting trees,

```
lgb_reg = lgb.LGBMRegressor(
    n_estimators=800,
    learning_rate=0.03,
    max_depth=-1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
```

One based on XGBoost,

```
xgb_reg = XGBRegressor(
    n_estimators=800,
    learning_rate=0.03,
    max_depth=8,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    tree_method="hist"
)
```

One based on CatBoost or a similar model.

```
cat_reg = CatBoostRegressor(
    depth=8,
    learning_rate=0.03,
    n_estimators=800,
    loss_function="RMSE",
    random_state=42,
    verbose=0
)
```

Each regressor was trained on the same spender subset and evaluated on a validation split using metrics such as mean absolute error (MAE):

```
lgb_reg.fit(Xr_train, yr_train)
xgb_reg.fit(Xr_train, yr_train)
cat_reg.fit(Xr_train, yr_train)

val_pred_lgb_log = lgb_reg.predict(Xr_val)
val_pred_xgb_log = xgb_reg.predict(Xr_val)
val_pred_cat_log = cat_reg.predict(Xr_val)

val_true = np.expm1(yr_val)
val_pred_lgb = np.expm1(val_pred_lgb_log)
val_pred_xgb = np.expm1(val_pred_xgb_log)
val_pred_cat = np.expm1(val_pred_cat_log)

val_pred_ens = (val_pred_lgb + val_pred_xgb + val_pred_cat) / 3.0

mae_lgb = mean_absolute_error(val_true, val_pred_lgb)
mae_xgb = mean_absolute_error(val_true, val_pred_xgb)
mae_cat = mean_absolute_error(val_true, val_pred_cat)
mae_ens = mean_absolute_error(val_true, val_pred_ens)
```

For inference, each regressor produced a prediction in log space; these were averaged and converted back to THB with the inverse log transform.

Combining both stages for final predictions

For each player in the test set:

```
# Stage 1: spender probability + best threshold
test_proba = clf.predict_proba(X_test)[:, 1]
test_is_spender = (test_proba > best_t).astype(int)

# Stage 2: ensemble prediction in log space
test_pred_lgb_log = lgb_reg.predict(X_test)
test_pred_xgb_log = xgb_reg.predict(X_test)
test_pred_cat_log = cat_reg.predict(X_test)

test_pred_lgb = np.expm1(test_pred_lgb_log)
test_pred_xgb = np.expm1(test_pred_xgb_log)
test_pred_cat = np.expm1(test_pred_cat_log)

test_pred_ens = (test_pred_lgb + test_pred_xgb + test_pred_cat) / 3.0

test_spend_final = np.where(test_is_spender == 1, test_pred_ens, 0.0)
```

The classifier produced a probability of being a spender and this was converted to a binary label using the optimal threshold found in the first stage.

The ensemble regressor produced a predicted log spending amount, which was then converted back to THB.

3.4 Evaluation & Results

The performance of the two-stage model was evaluated separately for the classification and regression parts.

Spender / non-spender classification:

```
[LightGBM] [Info] Number of positive: 43094, number of negative: 40106
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.043063 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4601
[LightGBM] [Info] Number of data points in the train set: 83200, number of used features: 32
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.517957 -> initscore=0.071858
[LightGBM] [Info] Start training from score 0.071858
Best threshold: 0.35
Best F1: 0.7410481384870761
```

On the validation set, scanning thresholds showed that a threshold lower than 0.5 (for example around 0.35) yielded the best F1 score.

The best F1 score was around the low-to-mid 0.7 range, indicating a reasonably good balance of precision and recall for identifying spenders.

This classifier's output directly controls how many players are treated as potential revenue contributors in the second stage.

Spending regression on spenders:

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.023979 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4591
[LightGBM] [Info] Number of data points in the train set: 43094, number of used features: 32
[LightGBM] [Info] Start training from score 7.939081
MAE LGBM: 3517.457211287501
MAE XGB: 3591.525904658227
MAE Cat: 3414.8206409032296
MAE Ensemble: 3462.6019639527008
NMAE Ensemble (approx): 0.16992499564893226
```

For the regression stage, performance was measured using mean absolute error (MAE) on the log-transformed target and then interpreted in the original THB scale.

Individually, the three regressors achieved MAE values that differed slightly, with the best model having the lowest MAE.

```
[LightGBM] [Info] Number of positive: 53868, number of negative: 50132
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.039502 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4614
[LightGBM] [Info] Number of data points in the train set: 104000, number of used features: 32
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.517962 -> initscore=0.071877
[LightGBM] [Info] Start training from score 0.071877
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.083855 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4600
[LightGBM] [Info] Number of data points in the train set: 53868, number of used features: 32
[LightGBM] [Info] Start training from score 7.942489
```

The simple average ensemble of the three models achieved an MAE better than or comparable to the best single model and a reasonable normalized MAE showing that averaging improves stability and robustness.

3.5 Insights gained from model behavior

Spending is highly concentrated: a relatively small group of players contributes most of the revenue. A single global regression model struggles to fit both the large zero-spending majority and the high-spending minority well.

Two-stage modeling improves performance: separating the “will spend?” decision from the “how much?” prediction gives more accurate and stable results for both parts.

Historical behavior is critical: features related to past spending and engagement (such as historical and previous-month spending, VIP level, premium membership, and event participation) have the strongest influence on future spending.

3.6 Common mistakes or failed experiments

Single-stage regression on all players: an early idea was to directly regress spending_30d on all players. This model performed poorly, especially on actual spenders, because the loss function was dominated by the many zero values.

No log transform on spending: training a regressor on the raw spending amounts led to unstable models that were heavily influenced by a few extremely high spenders.

Dropping rows with missing segment: removing all players with missing segment labels would have significantly reduced the training set size and lost valuable behavioral patterns.

3.7 Domain interpretation of the results

Short-term revenue forecasting: aggregating predicted 30-day spending across players provides an estimate of near-future revenue, which can be used in budgeting and planning.

Targeted promotions: the spender classifier identifies players with a high probability of spending; combined with predicted amounts, this helps prioritize which players should receive special offers, discounts, or exclusive bundles.

Retention and monetization strategy: players predicted to be borderline spenders (for example, moderate probability of spending and moderate predicted amounts) are prime candidates for experiments such as trial offers or time-limited events.

4. Task 4: Game Image Classification

4.1 EDA

Task 4 focuses on classifying game-related images into five categories based on their visual content.

Loaded the train, validation, and test CSV files:

```
DRIVE_ZIP_PATH = '/content/drive/MyDrive/ML_Project/task4.zip'
LOCAL_DIR = '/content/task4_local_data'

print("🚀 Initiating Grandmaster Pipeline...")
if not os.path.ismount('/content/drive'):
    drive.mount('/content/drive')
if not os.path.exists(LOCAL_DIR):
    print(f"⏳ Speed Boost: กำลัง Copy ไฟล์จาก {DRIVE_ZIP_PATH} ...")

    if not os.path.exists(DRIVE_ZIP_PATH):
        raise FileNotFoundError(f"❌ ไม่เจอไฟล์ {DRIVE_ZIP_PATH} ใน Drive! กรุณาเช็คชื่อไฟล์ครับ")

    os.system(f'cp "{DRIVE_ZIP_PATH}" /content/temp_data.zip')
    print("📦 Unzipping... (Fast SSD)")
    os.system(f'unzip -q /content/temp_data.zip -d "{LOCAL_DIR}"')
    os.system('rm /content/temp_data.zip')
    print(f"✅ Data Ready at: {LOCAL_DIR}")
else:
    print(f"⚡ Data Already Exists on SSD")

BASE_PATH = LOCAL_DIR
if not os.path.exists(os.path.join(BASE_PATH, 'train.csv')):
    for root, dirs, files in os.walk(LOCAL_DIR):
        if 'train.csv' in files:
            BASE_PATH = root
            break
```

```
TRAIN_CSV = f'{BASE_PATH}/train.csv'
VAL_CSV   = f'{BASE_PATH}/val.csv'
TEST_CSV  = f'{BASE_PATH}/test.csv'

TRAIN_IMG_DIR = f'{BASE_PATH}/train'
VAL_IMG_DIR   = f'{BASE_PATH}/val'
TEST_IMG_DIR  = f'{BASE_PATH}/test'
```

each containing at least:

A unique image identifier or file name.

A label column for train/validation (integer class IDs 0–4).

We confirmed the dataset is suitable for image classification, with moderate class imbalance and reasonably consistent image formats, but some potential missing or unreadable files that need handling in the data loader.

4.2 Data Preprocessing

The preprocessing pipeline for Task 4 is built around a custom PyTorch Dataset and standard image transforms.

Custom dataset design:

```
class GameDataset(Dataset):
    def __init__(self, csv_file, img_dir, transform=None, is_test=False):
        self.df = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.transform = transform
        self.is_test = is_test

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img_path = os.path.join(self.img_dir, row['file_name'])
        try:
            image = Image.open(img_path).convert("RGB")
        except:
            image = Image.new('RGB', (224, 224), color='black')

        if self.transform:
            image = self.transform(image)

        if self.is_test:
            return image, row['id']
        else:
            return image, int(row['label'])
```

A GameDataset class is used for both training/validation and test sets.

For each sample, the dataset

Takes an image file path from the dataframe.

Loads the image (e.g., via Pillow) and converts it to RGB.

Applies a specified transform pipeline (different for train vs. validation/test).

Returns:

For training/validation: (image_tensor, label).

For test: (image_tensor, image_id) for later submission.

If an image cannot be loaded (missing or corrupted), the dataset returns a fallback (e.g., a blank image) to avoid crashing training.

Data augmentation and normalization

Training transforms include:

```
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

Resizing/cropping to a fixed size expected by the backbone model.

Random horizontal flips and other light augmentations (e.g., random crops or slight rotations) to increase robustness.

Conversion to tensor and normalization using ImageNet-like mean and standard deviation.

Validation transforms are lighter:

```
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

Deterministic resize/center crop.

Conversion to tensor and the same normalization, without heavy augmentations, to evaluate true performance.

Test transforms mirror validation transforms, optionally with variations for test-time augmentation (see model design):

```
tta_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop((224, 224)), # สุ่ม Crop
    transforms.RandomHorizontalFlip(p=0.5), # สุ่มพลิก
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

Data loaders

Training, validation, and test datasets are wrapped in DataLoader objects with:

```
train_loader = DataLoader(GameDataset(TRAIN_CSV, TRAIN_IMG_DIR, transform=train_transform),
                          batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(GameDataset(VAL_CSV, VAL_IMG_DIR, transform=val_transform),
                        batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)
```

Appropriate batch size.

shuffle=True for training, shuffle=False for validation and test.

num_workers and ***pin_memory*** tuned for faster data loading.

This setup ensures efficient, batched loading of images during training and inference.

4.3 Model Design

The Task 4 model is based on a pretrained Swin Transformer (with a ResNet fallback) fine-tuned for a 5-class classification problem.

Backbone architecture:

```
print("🔨 Building Swin Transformer...")
try:
    model = models.swin_t(weights="DEFAULT")
    model.head = nn.Linear(model.head.in_features, NUM_CLASSES)
except:
    print("⚠️ Swin failed, using ResNet50")
    model = models.resnet50(weights="DEFAULT")
    model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)
```

The primary backbone is Swin Transformer Tiny, a hierarchical Vision Transformer architecture pretrained on ImageNet.

The original classification head is replaced with a new fully connected layer whose output dimension equals the number of classes (5).

If downloading the Swin weights fails, the notebook falls back to a ResNet-50 backbone, again replacing the final fully connected layer for 5-class output.

Loss function and training objective:

```
model = model.to(DEVICE)
optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=1e-2)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

scheduler = optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0=5, T_mult=2)
best_val_f1 = 0.0
```

The main loss used is cross-entropy, with label smoothing applied

Label smoothing reduces overconfidence and helps the model generalize better when labels may contain noise.

Optimizer and learning schedule

The optimizer is AdamW, suitable for transformer-based architectures, with:

A small learning rate for fine-tuning.

Weight decay to regularize the model.

A CosineAnnealingWarmRestarts scheduler is used to dynamically adjust the learning rate:

Starts with warm restarts every few epochs.

Gradually decreases the peak learning rate over time, helping avoid poor local minima and improving convergence.

Training loop:

```
for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")
    for images, labels in pbar:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        scheduler.step(epoch + pbar.n / len(train_loader))
        running_loss += loss.item()
        pbar.set_postfix({'loss': f"{loss.item():.4f}"})

    model.eval()
    preds, targs = [], []
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            _, p = torch.max(model(images), 1)
            preds.extend(p.cpu().numpy())
            targs.extend(labels.cpu().numpy())

    val_f1 = f1_score(targs, preds, average="macro")
    avg_loss = running_loss / len(train_loader)
    print(f"📊 Epoch {epoch+1}: Loss={avg_loss:.4f} | Val F1={val_f1:.4f}")

    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        torch.save(model.state_dict(), "best_model_task4.pth")
        print("⭐ New Best Model Saved!")
```

The model is trained for a fixed number of epochs (e.g., around 15), or until early stopping based on validation performance.

For each epoch, the loop:

Runs a training phase: forward pass, loss computation, backward pass, optimizer step, and scheduler step.

Runs a validation phase: computes loss and evaluation metrics on the validation set without gradient updates.

The best-performing model weights (based on validation metric) are saved to disk (e.g., best_model_task4.pth).

Test-Time Augmentation (TTA):

```
model.load_state_dict(torch.load("best_model_task4.pth"))
model.eval()
TTA_STEPS = 5
final_probs = None
test_ids = []
test_dataset_tta = GameDataset(TEST_CSV, TEST_IMG_DIR, transform=tta_transform, is_test=True)
test_loader_tta = DataLoader(test_dataset_tta, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)
for step in range(TTA_STEPS):
    print(f"    🔄 TTA Round {step+1}/{TTA_STEPS}...")

    step_probs = []
    current_ids = []

    with torch.no_grad():
        for images, ids in tqdm(test_loader_tta, leave=False):
            images = images.to(DEVICE)
            outputs = model(images)
            probs = torch.softmax(outputs, dim=1).cpu().numpy()
            step_probs.append(probs)
            if step == 0: current_ids.extend(ids)

    step_probs = np.concatenate(step_probs, axis=0)

    if final_probs is None:
        final_probs = step_probs
        test_ids = current_ids
    else:
        final_probs += step_probs

final_probs /= TTA_STEPS
final_preds = np.argmax(final_probs, axis=1)
```

For inference on the test set, TTA is used:

For each test image, multiple forward passes are run with slightly different test-time augmentations (e.g., different resized crops or flips).

The predicted class probabilities are averaged across these passes.

The final class prediction for each image is the argmax of the averaged probabilities.

This ensemble-like technique often yields a small but meaningful improvement in robustness and accuracy.

This model design leverages modern pretrained architectures, carefully chosen training strategies, and TTA to achieve strong performance on the image classification task.

4.4 Evaluation & Results

Evaluation focuses on performance on the validation set during training and uses that to select the final model for test predictions.

Validation metrics:

```
Epoch 1/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:28<00:00,  2.66it/s, loss=0.6361]
📊 Epoch 1: Loss=0.6742 | Val F1=0.7372
⭐ New Best Model Saved!
Epoch 2/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:32<00:00,  2.67it/s, loss=0.4124]
📊 Epoch 2: Loss=0.5172 | Val F1=0.7554
⭐ New Best Model Saved!
Epoch 3/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:31<00:00,  2.67it/s, loss=0.4402]
📊 Epoch 3: Loss=0.4730 | Val F1=0.7632
⭐ New Best Model Saved!
```

After each epoch, predictions on the validation set are collected and compared to ground truth labels.

Metrics computed include:

Accuracy (overall correct classification rate).

Macro F1 score (if implemented), which balances performance across all 5 classes.

The best epoch is tracked based on the chosen primary metric (e.g., highest macro F1 or highest accuracy).

Model selection

The model state that achieves the best validation performance is saved to disk.

At the end of training, this best checkpoint is reloaded for final inference, not necessarily the last epoch's model.

Final test predictions:

```
📊 Epoch 10: Loss=0.4325 | Val F1=0.7901
⭐ New Best Model Saved!
Epoch 11/15:  12% |▰▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱| 123/986 [00:49<05:45,  2.50it/s, loss=0.4053]
📊 Epoch 11: Loss=0.4245 | Val F1=0.7697
Epoch 12/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:32<00:00,  2.67it/s, loss=0.3995]
📊 Epoch 12: Loss=0.4153 | Val F1=0.7612
Epoch 13/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:32<00:00,  2.67it/s, loss=0.3924]
📊 Epoch 13: Loss=0.4093 | Val F1=0.7703
Epoch 14/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:32<00:00,  2.67it/s, loss=0.3927]
📊 Epoch 14: Loss=0.4064 | Val F1=0.7746
Epoch 15/15: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 986/986 [06:32<00:00,  2.67it/s, loss=0.4248]
📊 Epoch 15: Loss=0.4042 | Val F1=0.7736

🔮 Predicting with Test-Time Augmentation (TTA)...
  🔄 TTA Round 1/5...

  🔄 TTA Round 2/5...

  🔄 TTA Round 3/5...

  🔄 TTA Round 4/5...

  🔄 TTA Round 5/5...
```

With the best model reloaded, TTA is used to obtain robust class probabilities for all test images.

For each image, the predicted label is the class with highest averaged probability.

4.5 Insights gained from model behavior

Pretrained transformers are powerful: starting from a pretrained Swin backbone allowed the model to reach high performance in relatively few epochs compared with training a CNN from scratch.

Label smoothing improves robustness: smoothing the labels helped mitigate overfitting, especially when the training set was not extremely large or when some labels might be ambiguous.

Augmentations matter: modest data augmentation during training and TTA during inference both contributed to more stable performance across different types of images and viewpoints.

4.6 Common mistakes or failed experiments

No or insufficient augmentation: experiments with very weak augmentation led to overfitting, as the model memorized training images but generalized poorly to validation images.

Using too high a learning rate for fine-tuning: larger learning rates sometimes caused unstable training or degraded pretrained feature representations.

Training from scratch: a purely randomly initialized model performed much worse and required far more compute to approach the same metrics.

4.7 Domain interpretation of the results

Asset organization and tagging: automatically classifying images into different gameplay or UI categories allows the content pipeline to stay organized as the asset library grows.

Content analytics: understanding which categories of images are most frequently produced or used can provide insights into design trends and player-facing content emphasis.

Quality control: automated checks on screenshots or promotional images can help verify that assets belong to the correct category, catching mismatches or outdated visuals before release.

5. Task 5: Account Security Monitoring (Anomaly Detection)

5.1 EDA

Task 5 aims to flag suspicious player accounts or sessions as anomalies using only behavioral data (no labels). The dataset consists of many numeric features, often arranged in groups that look like time steps (for example, **feature_x_1**, **feature_x_2**, **feature_x_3**, **feature_x_4**), which appear to capture the same metric at different points in time.

Loaded the dataset and examined:

```
def run_task5_pipeline():
    print("--- Starting Task 5: Account Security Monitoring (Ensemble) ---")
    try:
        df = pd.read_csv('test.csv')
        print(f"Data Loaded. Shape: {df.shape}")
    except FileNotFoundError:
        print("Error: 'task5/test.csv' not found. Please ensure the file exists.")
        return
```

Number of rows (players/sessions) and number of columns (features):

The presence of an id column used to identify accounts in the final submission:

```
player_ids = df['id'] if 'id' in df.columns else df.index
features = df.drop(columns=['id'], errors='ignore')
```

Inspected the distribution of values for several representative features:

Detected skewed distributions, with most players in a normal range and a few extreme outliers.

Observed that many features came in groups of four, suggesting a temporal or multi-snapshot structure.

Checked for missing values across the feature set and confirmed that some features contain NaNs, which must be handled before fitting distance- or density-based anomaly detectors.

From this initial analysis we concluded that the data is high-dimensional, mostly numeric, and likely contains both “stable” players and a relatively small number of strange or extreme behaviors we want to detect as anomalies.

5.2 Data Preprocessing

Because anomaly detection algorithms are sensitive to scaling and missing values, preprocessing for Task 5 focuses on cleaning and summarizing the raw features, especially the time-like groups.

ID preservation and feature selection:

```
base_features = set()
for col in features.columns:
    if col[-2:] in ['_1', '_2', '_3', '_4']:
        base_features.add(col[:-2])

for base in base_features:
    cols = [f"{base}_{i}" for i in range(1, 5)]
    if all(c in features.columns for c in cols):
        features[f'{base}_std'] = features[cols].std(axis=1)
        features[f'{base}_mean'] = features[cols].mean(axis=1)
        features[f'{base}_range'] = features[cols].max(axis=1) - features[cols].min(axis=1)

features = features.fillna(0)
```

The id column is kept separately as *player_ids* for later use in the submission file.

All remaining columns (excluding the ID) are treated as candidate features for anomaly detection.

Time-series style aggregation

Many columns share a common base name with suffixes such as _1, _2, _3, _4. These are interpreted as the same metric measured at four different times or conditions.

For each such base feature:

The four related columns are grouped together.

Three aggregate statistics are computed across the four values:

Mean over time (average behavior),

Standard deviation (volatility or variability),

Range (max − min, capturing fluctuation amplitude).

These aggregated features are then added to the feature set. They summarize how stable or erratic a player's behavior is over these time steps, which is informative for anomaly detection.

Handling missing values. All remaining missing values in the feature matrix are filled with zeros (or an equivalent neutral value) to ensure that no NaNs are passed to the anomaly detection models.

Feature scaling:

```
numeric_features = features.select_dtypes(include=np.number)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(numeric_features)
```

Because models such as One-Class SVM and Local Outlier Factor rely on distances in feature space, all numeric features (including aggregates) are standardized using a standard scaler (zero mean, unit variance).

This prevents high-variance or large-scale features from dominating the anomaly score.

After preprocessing, we obtain a clean, scaled feature matrix suitable for multiple unsupervised anomaly detection algorithms.

5.3 Model Design

Task 5 is formulated as an unsupervised anomaly detection problem: there are no explicit labels telling us which accounts are fraudulent or compromised. To improve robustness, we use an ensemble of three different detectors that each define “anomaly” in a different way.

Isolation Forest:

```
contamination_rate = 0.05
```

```
print("1. Training Isolation Forest...")
iso_forest = IsolationForest(
    n_estimators=200,
    contamination=contamination_rate,
    random_state=42,
    n_jobs=-1
)
# Output: -1 (Anomaly), 1 (Normal)
pred_iso = iso_forest.fit_predict(X_scaled)
```

A tree-based model that randomly partitions the feature space:

Anomalies are expected to be isolated with fewer splits (shorter paths).

Key characteristics:

Works well in high dimensions and scales to large datasets.

Uses a contamination parameter (e.g., around 5%) to estimate the proportion of anomalies.

One-Class SVM:

```
print("2. Training One-Class SVM...")
oc_svm = OneClassSVM(
    kernel='rbf',
    nu=contamination_rate,
    gamma='scale'
)
pred_svm = oc_svm.fit_predict(X_scaled)
```

A kernel-based method that learns a “boundary” around the majority of the data:

Points lying outside this learned region are considered anomalies.

Uses an RBF kernel with parameters like nu (an upper bound on the fraction of anomalies) and gamma (kernel width).

Also outputs labels -1 (anomaly) and 1 (normal).

Local Outlier Factor (LOF):

```
print("3. Training Local Outlier Factor...")
lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination=contamination_rate,
    n_jobs=-1,
    novelty=False # LOF is strictly outlier detection here
)
pred_lof = lof.fit_predict(X_scaled)
```

A density-based method:

Compares the local density around each point to the density around its neighbors.

Points that are much less dense than their neighbors receive high outlier scores.

Uses parameters such as number of neighbors and contamination fraction.

Produces -1 for outliers and 1 for inliers.

Ensemble and voting scheme:

```
res_iso = np.where(pred_iso == -1, 1, 0)
res_svm = np.where(pred_svm == -1, 1, 0)
res_lof = np.where(pred_lof == -1, 1, 0)
total_votes = res_iso + res_svm + res_lof
final_preds = np.where(total_votes > 1, 1, 0)
```

Each of the three models is trained on the preprocessed feature matrix (unsupervised).

5.4 Evaluation & Results

Because Task 5 is unsupervised and we have no ground-truth labels, evaluation is mainly descriptive and based on model agreement and the proportion of flagged accounts:

```
--- Starting Task 5: Account Security Monitoring (Ensemble) ---
Data Loaded. Shape: (25889, 124)
Engineering features...
Scaling features...
Training Ensemble Models (Contamination: 0.05)...
1. Training Isolation Forest...
2. Training One-Class SVM...
3. Training Local Outlier Factor...
Aggregating votes...

--- Ensemble Prediction Summary ---
Model Agreement Breakdown:
0 Votes (Normal)            23500
1 Vote                       1089
2 Votes                      1108
3 Votes (Strong Anomaly)      192
Name: count, dtype: int64

Final Decision (Majority Vote):
is_anomaly
0    24589
1     1300
```

Model voting distribution

After running all three detectors:

Many accounts receive 0 votes (no model thinks they are anomalous).

A smaller number have 1 vote (only one model thinks they are suspicious).

A smaller group have 2 or 3 votes, where multiple models agree they are outliers.

Final anomaly rate

Applying the majority rule threshold (total_votes > 1) yields:

Most players are labeled normal (is_anomaly = 0).

A relatively small fraction (e.g., around 5% of the population) are labeled as anomalies (is_anomaly = 1).

This anomaly rate is consistent with typical expectations in security and fraud scenarios, where only a small portion of the population should be flagged for review.

5.5 Insights gained from model behavior

Isolation Forest tends to flag points that are globally “isolated” in the feature space.

One-Class SVM is sensitive to the overall shape of the data manifold and can pick up subtle boundary anomalies.

LOF is focused on local density, flagging points that are sparse compared to their neighbors.

Some accounts are flagged by only one method; many of these may be borderline or noisy cases.

Requiring agreement from at least two models increases confidence that flagged accounts are genuinely unusual.

5.6 Common mistakes or failed experiments

Relying on just one detector (e.g., only Isolation Forest) made the results sensitive to that model’s hyperparameters and assumptions about the data.

Setting the contamination fraction too high produced far too many anomalies, overwhelming any realistic review process.

Setting it too low risked missing meaningful suspicious behavior.

Early attempts without feature scaling caused One-Class SVM and LOF to behave poorly, as some features dominated the distance computations.

Marking an account as anomalous if even one model said so resulted in a large number of false positives. Requiring majority agreement provided a better balance and a cleaner signal.

5.7 Domain interpretation of the results

The accounts flagged as is_anomaly = 1 form a prioritized list for human analysts or customer support to investigate:

Sudden spikes in activity,

Unusual patterns across time steps,

Behavior that deviates strongly from typical players.

Regularly running this model on new behavioral logs can act as an early warning system for:

Account takeovers (compromised accounts exhibiting abnormal activity),

Bots or scripts that generate repetitive but unusual metrics,

Fraud or exploitation of game systems.

Once some anomalies are investigated and confirmed (true fraud vs false alarm), these outcomes can be used to create labeled datasets for future supervised fraud models.

Team Collaboration

Member:

Nattanan Charukornkul 66070501017

Project Coordinator, and Task 5

Titi Tiyachaipanich 66070501026

Project Coordinator, and Tester

Ratchanon Tiptamool 66070501045

Task 2, Task 4, and Code Refiner

Thammapon Chaengpatee 66070501076

Task 1, Task 3, and Report Writer

1. Planning and task allocation

At the beginning of the project, we discussed the overall scope of the five tasks and agreed on a rough division of responsibilities while still keeping every member involved in all tasks at some level.

Even with this specialization, we made sure that all members understood the main idea behind every task, so that the final report would reflect a shared understanding rather than five separate mini-projects.

2.Communication and problem solving

We communicated regularly through online chat and short calls. When a model did not train properly (e.g., unstable loss, bad F1 score), we discussed possible reasons together and tried alternative settings.

When new feature ideas were proposed (for example, two-stage modeling for Task 3 or aggregate time-series statistics for Task 5), we evaluated the complexity vs. benefit as a team before implementing them.

Conclusion

1.Key Lesson

This project covered five different but related machine learning tasks in a gaming context: cheater detection, player segmentation, short-term spending prediction, image classification, and anomaly detection. Together, these tasks form a small but realistic analytics pipeline for a game company.

We learned that data preprocessing and feature engineering are just as important as the choice of model. Handling missing values carefully, log-transforming skewed variables, and designing domain-specific features significantly improved model performance in Tasks 1–3.

We saw the importance of handling imbalance and choosing appropriate metrics. For example, in Tasks 1 and 3, optimizing for F1 score and carefully tuning probability thresholds gave more useful results than simply maximizing accuracy.

We practiced multi-stage modeling, such as treating “will spend?” and “how much?” as separate problems in Task 3, and saw how this approach can better reflect the real structure of the data.

We gained experience with modern deep learning architectures for images (Task 4), including the use of pretrained backbones, label smoothing, and test-time augmentation to improve robustness.

We explored unsupervised anomaly detection (Task 5) and understood how combining different detectors with an ensemble/voting scheme can yield a practical list of suspicious accounts even without labeled data.

2.Insights on applying ML to the game industry

Cheater detection (Task 1) and behavioral anomaly detection (Task 5) can protect honest players, reduce abuse, and detect account takeovers or bots. These models act as an automated “early warning system” that can prioritize cases for manual review.

Player segmentation (Task 2) turns raw behavior logs into interpretable groups – for example, heavy spenders, grinders, social players, and casuals. Combined with short-term spending prediction (Task 3), this supports personalized offers, retention campaigns, and better live-ops planning.

Image classification (Task 4) can tag screenshots or assets by type, which helps organize content pipelines and could be extended to recommend skins/maps, check UI consistency, or auto-label marketing materials.

Together, these models provide signals that different teams can use: security and operations (Tasks 1 & 5), product and marketing (Tasks 2 & 3), and art/content teams (Task 4). This illustrates how ML is not a single model but a collection of tools supporting different parts of the game lifecycle.

3 Suggestions for future improvements or extensions

Use tools such as SHAP or feature importance dashboards so designers and business stakeholders can understand why the models make particular predictions (e.g., which behaviors trigger cheater flags or high LTV).

Replace simple aggregate features with sequence models (RNNs, temporal transformers) that consume full time-series data for behavior and spending, potentially improving Tasks 1, 3, and 5.

Player behavior and game meta change over time. Periodic retraining or online learning pipelines would help keep the models up-to-date and reduce performance decay.

Deploy the models as services feeding a shared dashboard where game designers, analysts, and operations staff can monitor KPIs, inspect individual players, and trigger actions (e.g., send offers, review flagged accounts).

Use A/B experiments to measure the real business impact of model-driven decisions (for example, targeted promotions based on Task 3 predictions) and feed outcomes back into the models as additional training data.